Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
- Αλγόριθμοι ευρετικής αναζήτησης
  - Αναζήτηση Πρώτα στο Καλύτερο (BestFS)
  - Αναρρίχηση Λόφων (HC)
  - Αλγόριθμος A*
- Παιχνίδια δύο αντιπάλων

Τυφλή και ευριστική αναζήτηση

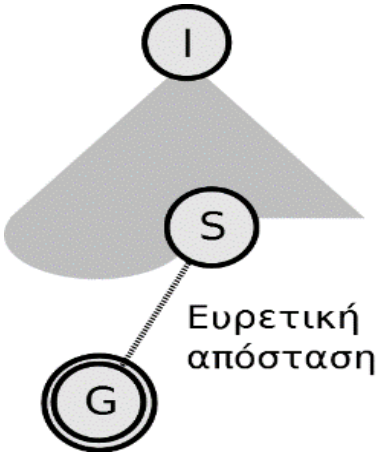
- Αλγόριθμοι **τυφλής αναζήτησης**:
  - Καμία πληροφορία για το αν το μονοπάτι που ακολουθούν τους οδηγεί σε λύση.
  - π.χ. DFS, BFS, ID.
- Αλγόριθμοι **ευριστικής αναζήτησης**:
  - Η αναζήτηση καθοδηγείται από ευριστικό μηχανισμό, που **αξιολογεί την κάθε δυνατή κατεύθυνση**.
  - Ο ευριστικός μηχανισμός ενσωματώνει **γνώση για το συγκεκριμένο πρόβλημα**.
  - Προτιμούμε **κατευθύνσεις** με **υψηλή προβλεπόμενη αξία**.

Αλγόριθμοι Ευρετικής Αναζήτησης (Heuristic Search Algorithms)

- Ευρετικός μηχανισμός (heuristic) είναι μία **στρατηγική**, βασισμένη στη γνώση για το συγκεκριμένο πρόβλημα, η οποία χρησιμοποιείται ως βοήθημα στη γρήγορη επίλυσή του.
- Ο ευρετικός μηχανισμός υλοποιείται συνήθως με **ευρετική συνάρτηση** (heuristic function), που έχει πεδίο ορισμού το σύνολο των καταστάσεων ενός προβλήματος και πεδίο τιμών το σύνολο τιμών που αντιστοιχεί σε αυτές.
- **Ευρετική τιμή** (heuristic value) είναι η τιμή της ευρετικής συνάρτησης για μια συγκεκριμένη κατάσταση του προβλήματος και εκφράζει το πόσο κοντά βρίσκεται η κατάσταση αυτή σε μία τελική.
- Η ευρετική τιμή δεν είναι η πραγματική τιμή της απόστασης από μία τερματική κατάσταση, αλλά μία **εκτίμηση** (estimate) που πολλές φορές μπορεί να είναι και λανθασμένη.

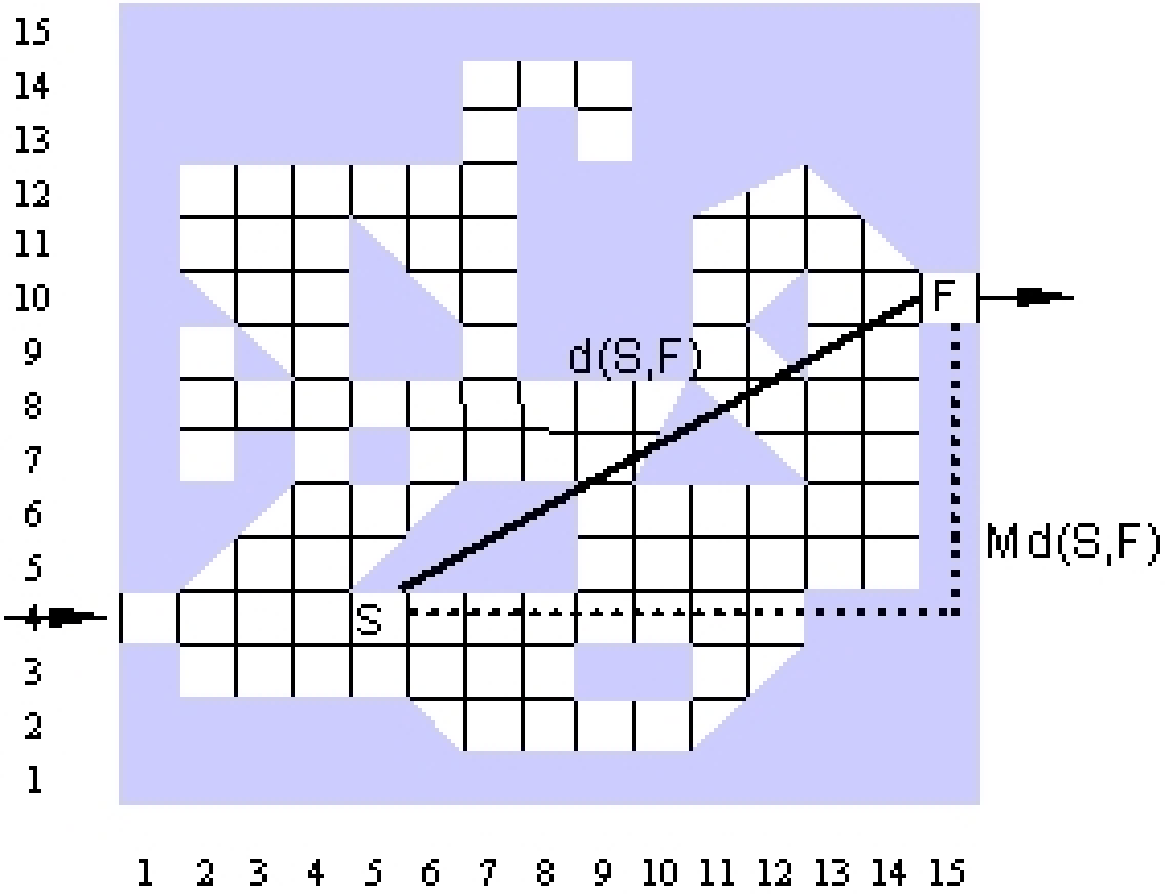
Ευριστικός μηχανισμός

- Συνήθως μια **συνάρτηση** h: S → R, που αξιολογεί κάθε κατάσταση.
  - **h(s)**: **εκτίμηση της απόστασης** της s από την πλησιέστερη **τελική κατάσταση**.
  - Οι εκτιμήσεις είναι **προσεγγιστικές** και δεν οδηγούν πάντα στο σωστό συμπέρασμα.
- Γενική ιδέα: **επεκτείνουμε** την κατάσταση s του μετώπου αναζήτησης με το **μικρότερο h(s)**.



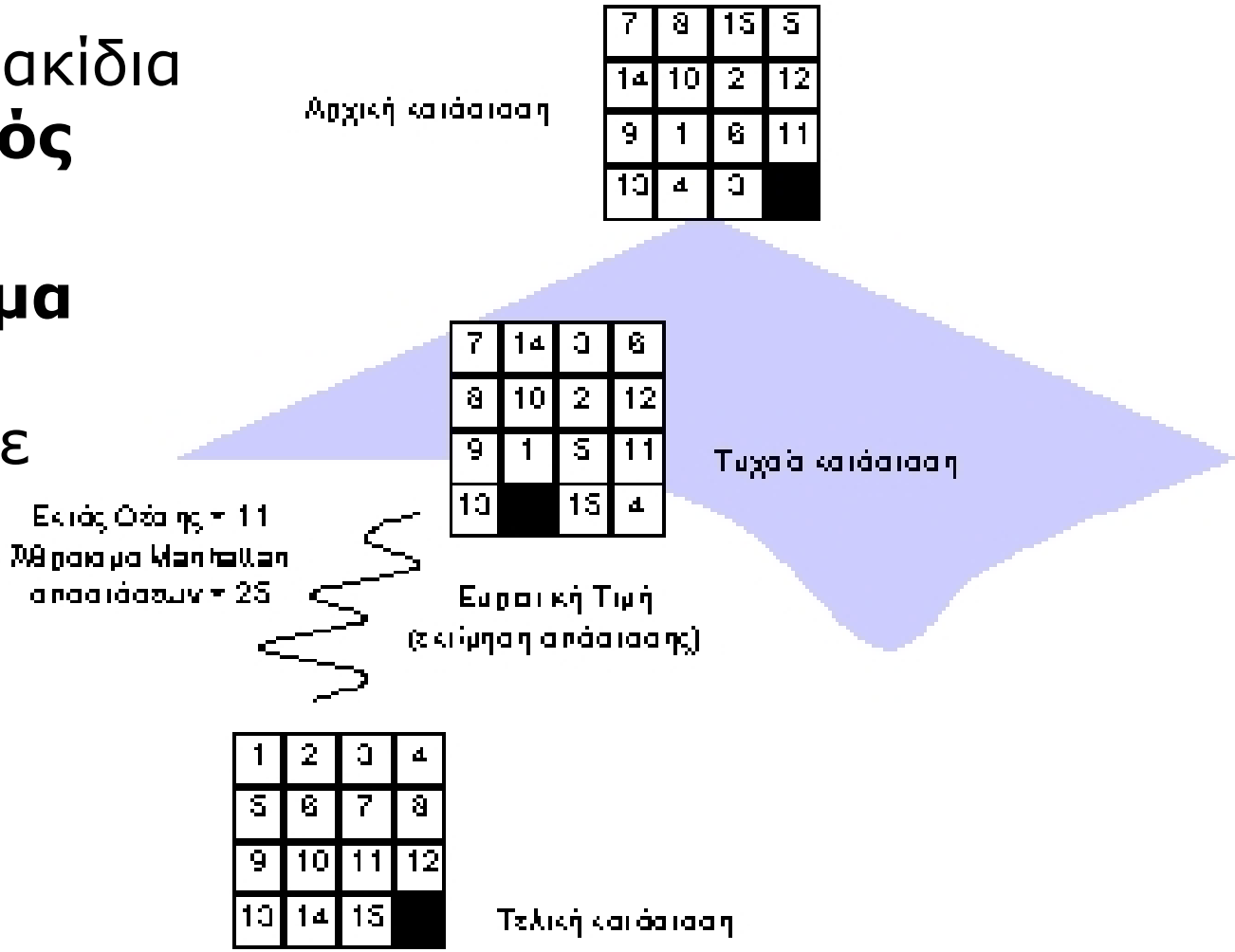
Ευριστικές συναρτήσεις

- Ευκλίδεια απόσταση: $d(S,F) = \sqrt{(X_S - X_F)^2 + (Y_S - Y_F)^2}$
- Απόσταση Manhattan: $Md(S,F) = |X_S - X_F| + |Y_S - Y_F|$



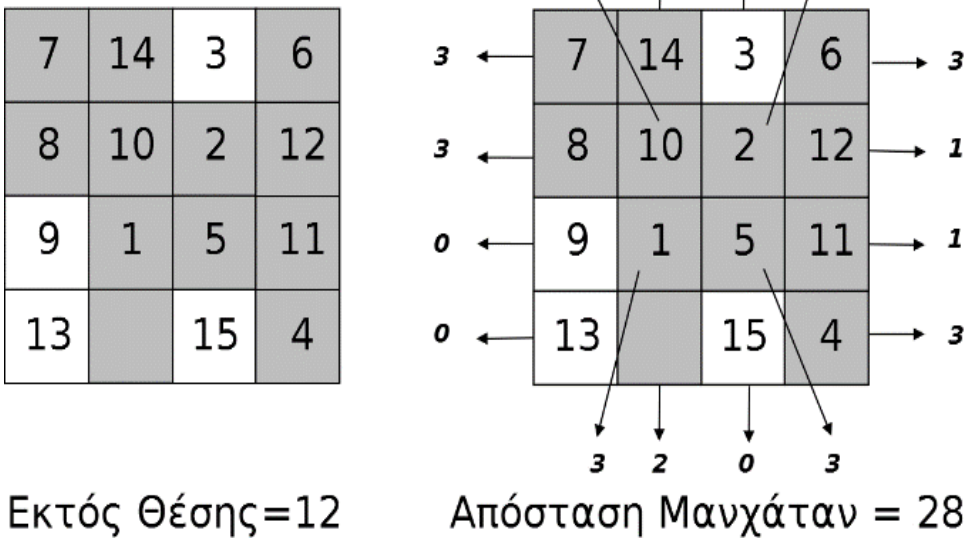
Ευριστικές συνάρτησεις

- h1(s): πόσα πλακίδια βρίσκονται **εκτός θέσης**;
- h2(s): **άθροισμα αποστάσεων** Manhattan κάθε πλακιδίου.



Ευριστικές συνάρτησεις

- h1(s): πόσα πλακίδια βρίσκονται **εκτός θέσης**;
- h2(s): **άθροισμα αποστάσεων** Manhattan κάθε πλακιδίου.

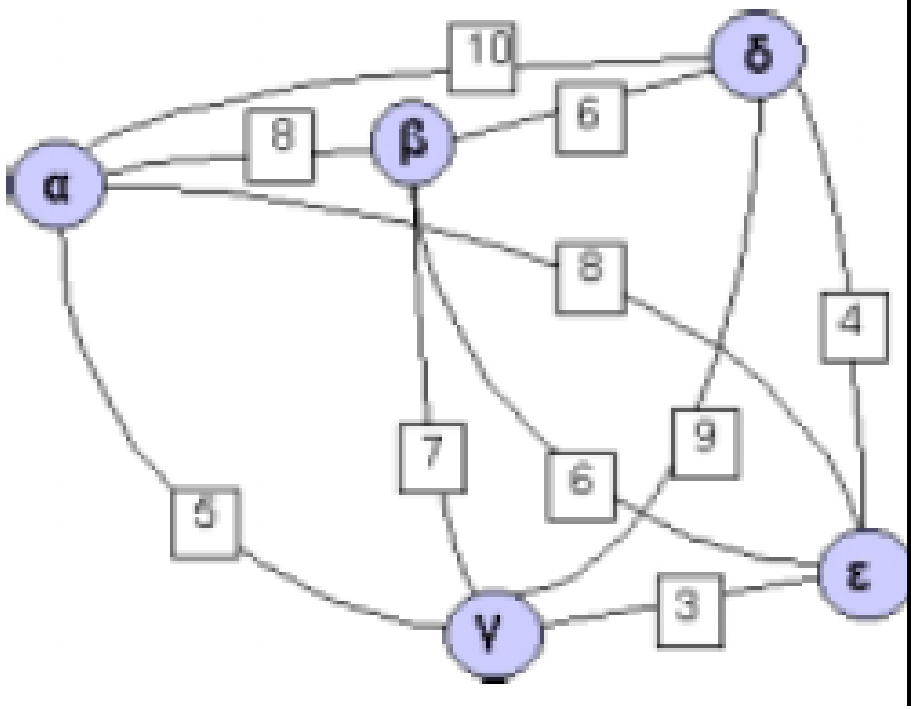


Εκτός Θέσης=12

Απόσταση Μανχάταν = 28

Ευριστικές συνάρτησεις

- Ευρετικός μηχανισμός (όχι συνάρτηση) στο πρόβλημα TSP
- Η κοντινότερη πόλη έχει κάθε φορά περισσότερες πιθανότητες να οδηγήσει σε μία καλή λύση.
- Ακολουθώντας τον παραπάνω μηχανισμό και ξεκινώντας από την α, θα επιλέγαμε: **α→γ→ε→δ→β→α** με κόστος 26.



Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
- Αλγόριθμοι ευρετικής αναζήτησης
  - Αναζήτηση Πρώτα στο Καλύτερο (BestFS)
  - Αναρρίχηση Λόφων (HC)
  - Αλγόριθμος A*
- Παιχνίδια δύο αντιπάλων

Αναζήτηση Πρώτα στο Καλύτερο (Best-First Search, BestFS)

- **Γενική ιδέα:** Οι καταστάσεις στο μέτωπο αναζήτησης ταξινομούνται βάση της τιμής της ευρετικής συνάρτησης, ενώ σε κάθε επανάληψη επιλέγεται αυτή με την μικρότερη ευρετική τιμή.
- Πλεονεκτήματα:
  - Προσπαθεί να δώσει μια γρήγορη λύση σε κάποιο πρόβλημα. Το αν τα καταφέρει ή όχι εξαρτάται πολύ από τον ευρετικό μηχανισμό.
  - Είναι πλήρης.
- Μειονεκτήματα:
  - Το μέτωπο αναζήτησης μεγαλώνει με υψηλό ρυθμό και μαζί του ο χώρος που χρειάζεται για την αποθήκευσή του, καθώς και ο χρόνος για την επεξεργασία των στοιχείων του.
  - Δεν εγγυάται ότι η λύση που θα βρεθεί είναι η βέλτιστη.

Χαρακτηριστικά BestFS

- **Αναδιάταξη μετώπου** σύμφωνα με τις τιμές της ευριστικής συνάρτησης.
- Προσπαθεί να ακολουθήσει **πρώτα το πιο ελπιδοφόρο** μονοπάτι.
- Σταματά στην πρώτη λύση. Μπορει να τροποποιηθεί ώστε να βρίσκει όλες τις λύσεις (οπότε **γίνεται πλήρης**) αλλά χάνεται το πλεονέκτημα της γρήγορης λύσης.
- Δεν εγγυάται ότι η (πρώτη) λύση θα είναι βέλτιστη (**μη αποδεκτός**, αν σταματάω στην πρώτη λύση).

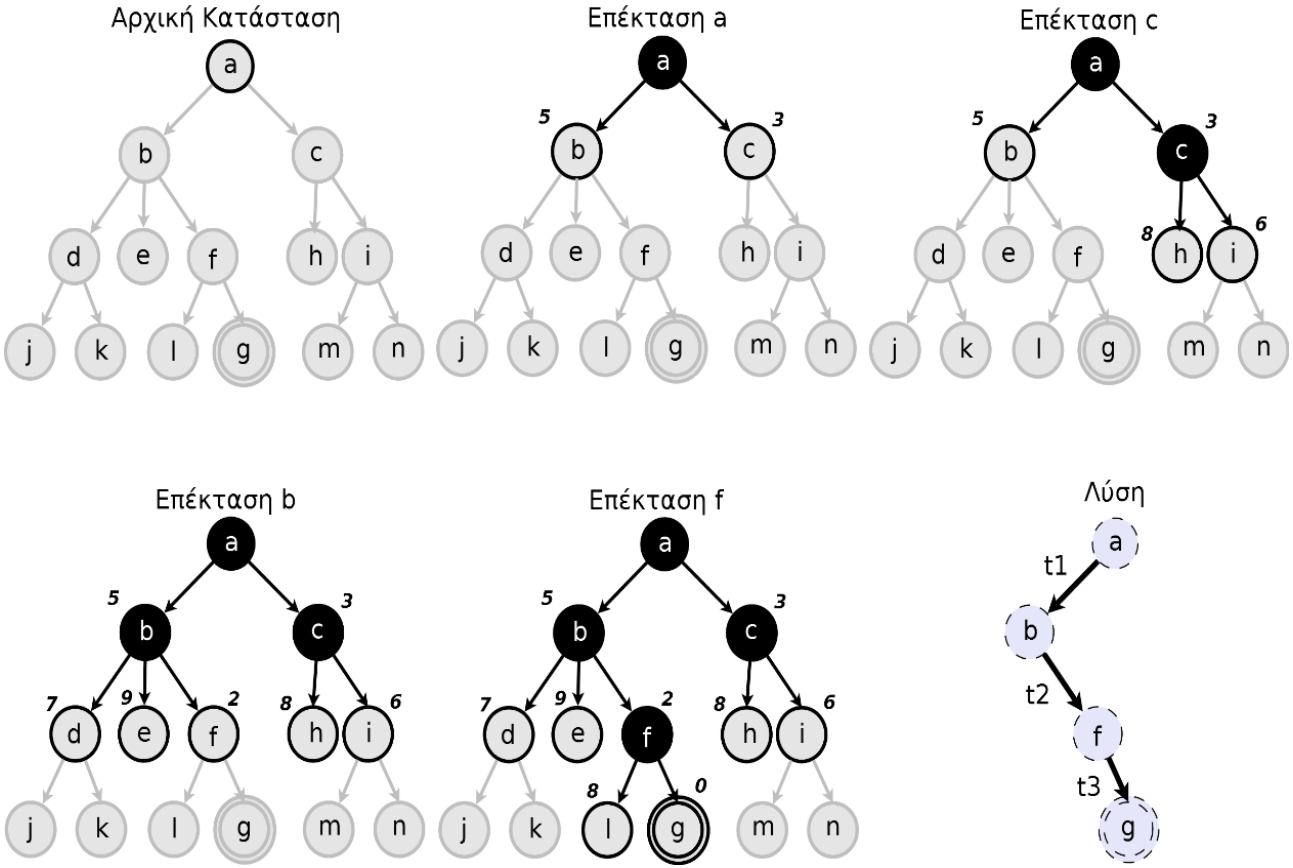
Πρώτα στο καλύτερο (BestFS)

1. Βάλε τη **ρίζα** (αρχική κατάσταση) στο μέτωπο αναζήτησης.
2. Αν το **μέτωπο είναι άδειο**, σταμάτησε.
3. Βγάλε τον **πρώτο σε σειρά** κόμβο από το μέτωπο.
4. Αν ο κόμβος ανήκει στο **κλειστό σύνολο**, πήγαινε στο βήμα 2. Διαφορετικά πρόσθεσέ τον στο κλειστό σύνολο.
5. Αν ο κόμβος αντιστοιχεί σε **τελική κατάσταση**, τύπωσε τη λύση και σταμάτησε.
6. **Επέκτεινε** τον κόμβο και **πρόσθεσε τα παιδιά στο μέτωπο αναζήτησης**.
7. **Αναδιάταξε το μέτωπο αναζήτησης** σύμφωνα με την ευριστική συνάρτηση, ώστε η καλύτερη κατάσταση να βρίσκεται στην αρχή του μετώπου.
8. Πήγαινε στο βήμα 2

Πρώτα στο καλύτερο (BestFS)

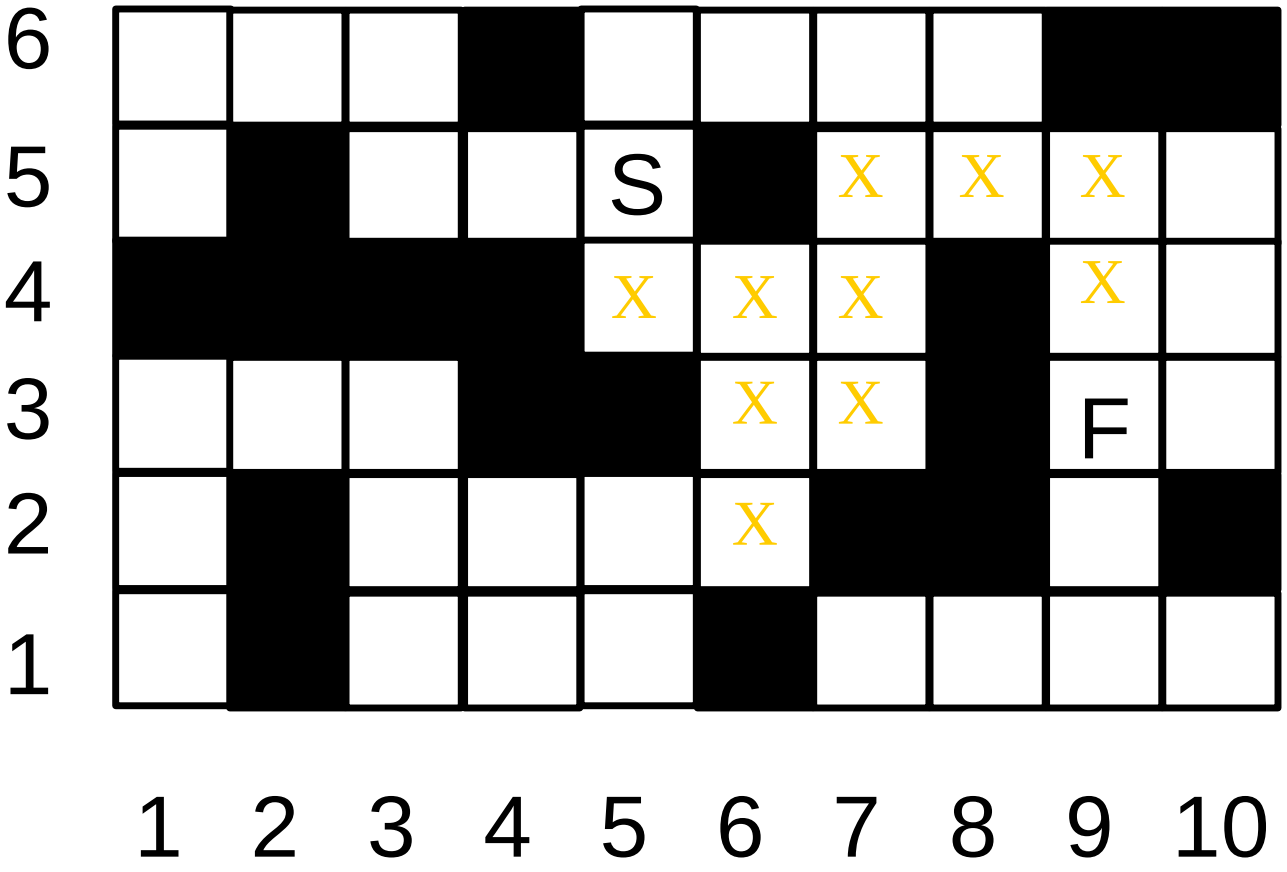
```
algorithm bestfs(InitialState, FinalStates)
        begin
                Closed ← ∅;
                EvaluatedInitialState ← Heuristic(<InitialState>)
                Frontier ← <EvaluatedInitialState>;
                CurrentState  ← best(Frontier);
                while CurrentState ∉ FinalStates do
                        Frontier ← delete(CurrentState,Frontier);
                        if CurrentState ∉ ClosedSet then
                        begin
                                Children ←  Expand(CurrentState);
                                EvaluatedChildren ← Heuristic(Children);
                                Frontier ← Frontier ^ EvaluatedChildren;
                                Closed ← Closed{CurrentState};
                        end;
                        if Frontier= ∅ then return fail;
                        CurrentState ← best(Frontier);
                endwhile;
                return success;
        end.
```

Πρώτα στο καλύτερο (BestFS)



Αναζήτηση Πρώτα στο Καλύτερο σε χώρο αναζήτησης με αρχική κατάσταση *a* και τελική *g*.

Αναζήτηση με BestFS στο πρόβλημα λαβύρινθου

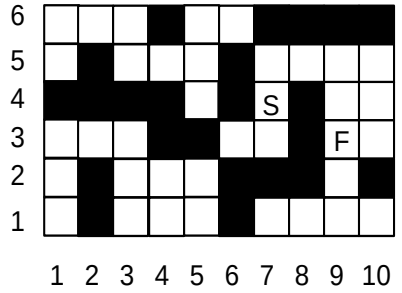


Η ευρεστική συνάρτηση υπολογίζει την απόσταση Manhattan

Μέτωπο	Κ.Σ.	Κατ/ση	Παιδιά
5-5	{}	5-5	5-4:5, 5-6:7, 4-5:7
5-4:5, 5-6:7, 4-5:7	5-5	5-4	5-5:6, 6-4:4
6-4:4, 5-5:6, 5-6:7, 4-5:7	+5-4	6-4	5-4:7, 6-3:3, 7-4:3
6-3:3, 7-4:3, 5-5:6, …	+6-4	6-3	6-4:4, 6-2:3, 7-3:2
7-3:2, 6-2:3, 7-4:3, …	+6-3	7-3	6-3:3, 7-4:4
6-3:3, 6-2:3, 7-4:3, …	+7-3	6-3	(βρόχος)
6-2:3, 7-4:3, 6-4:4, …	(ίδιο)	6-2	5-2:5, 6-3:3
7-4:3, 6-4:4, 5-2:5, …	+6-2	7-4	7-5:4, 6-4:4, 7-3:2
7-3:2, 7-5:4, 6-4:4, …	+7-2	7-3	(βρόχος)
7-5:4, 6-4:4, 5-2:5, …	(ίδιο)	7-5	7-4:3, 8-5:3, 7-6:5
8-5:3, 7-4:3, 6-4:4, …	+7-5	8-5	8-6:4, 7-5:4, 9-5:2
9-5:2, 7-4:3, 6-4:4, …	+8-5	9-5	8-5:3, 9-4:1
9-4:1, 8-5:3, 7-4:3, …	+9-5	9-4	9-3:0, 9-5:2, 10-4:2
9-3:0, 9-5:2, 10-4:2, …	+9-4	9-3	(τελική)

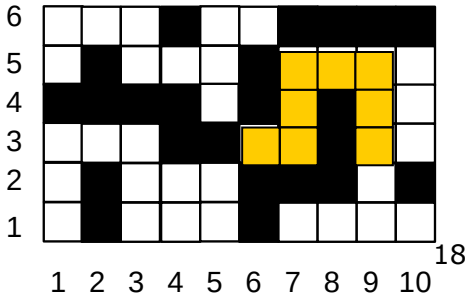
Πρόβλημα

- Στο λαβύρινθο επιτρέπονται μόνο οι κινήσεις προς τα επάνω, κάτω, δεξιά και αριστερά (όχι διαγωνίως). Χρησιμοποιούμε ως ευριστική συνάρτηση την απόσταση Μανχάταν (|X1 – X2| + |Y1 – Y2|) και προσπαθούμε να φτάσουμε από το S στο F με αναζήτηση πρώτα του καλύτερου (best first search).
- Συμπληρώστε τον παρακάτω πίνακα με τα βήματα που θα εκτελέσει ο αλγόριθμος.
- Μέτωπο αναζήτησης (X-Y:ευριστική απόσταση) κλειστό σύνολο τρέχουσα κατάσταση παιδιά που δεν ανήκουν στο κλειστό σύνολο 7-4:3 {} 7-4 7-3:2, 7-5:4



Πρόβλημα

Μέτωπο Αναζήτησης	Κλειστό Σύνολο	Τρέχουσα Κατάσταση	Παιδιά που δεν ανήκουν στο κλειστό σύνολο
7-4:3	{}	7-4	7-3:2, 7-5:4
7-3:2, 7-5:4	{7-4}	7-3	6-3:3
6-3:3, 7-5:4	{7-4, 7-3}	6-3	-
7-5:4	{7-4, 7-3, 6-3}	7-5	8-5:3
8-5:3	{7-4, 7-3, 6-3, 7-5}	8-5	9-5:2
9-5:2	{7-4, 7-3, 6-3, 7-5, 8-5}	9-5	9-4:1, 10-5:3
9-4:1, 10-5:3	{7-4, 7-3, 6-3, 7-5, 8-5, 9-5}	9-4	9-3:0, 10-4:2
9-3:0, 10-4:2, 10-5:3	{7-4, 7-3, 6-3, 7-5, 8-5, 9-5, 9-4}	9-3	τελική



Παράδειγμα BestFS σε N-puzzle

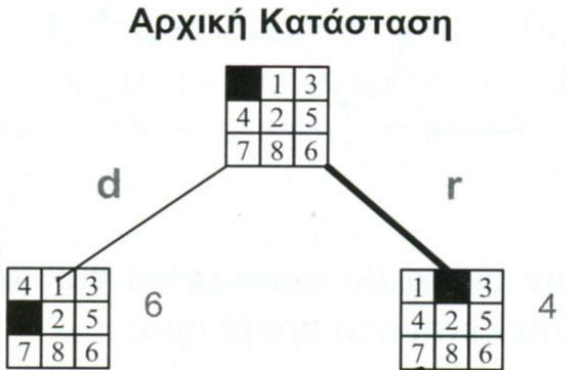
Στο πρόβλημα του puzzle.

• η βασική προϋπόθεση είναι ότι η κενή θέση μένει μέσα στα όρια του puzzle.



Οι τελεστές μετάβασης είναι :

- Η άδεια θέση κινείται αριστερά **l**
- Η άδεια θέση κινείται κάτω **d**
- Η άδεια θέση κινείται δεξιά **r**
- Η άδεια θέση κινείται πάνω **u**

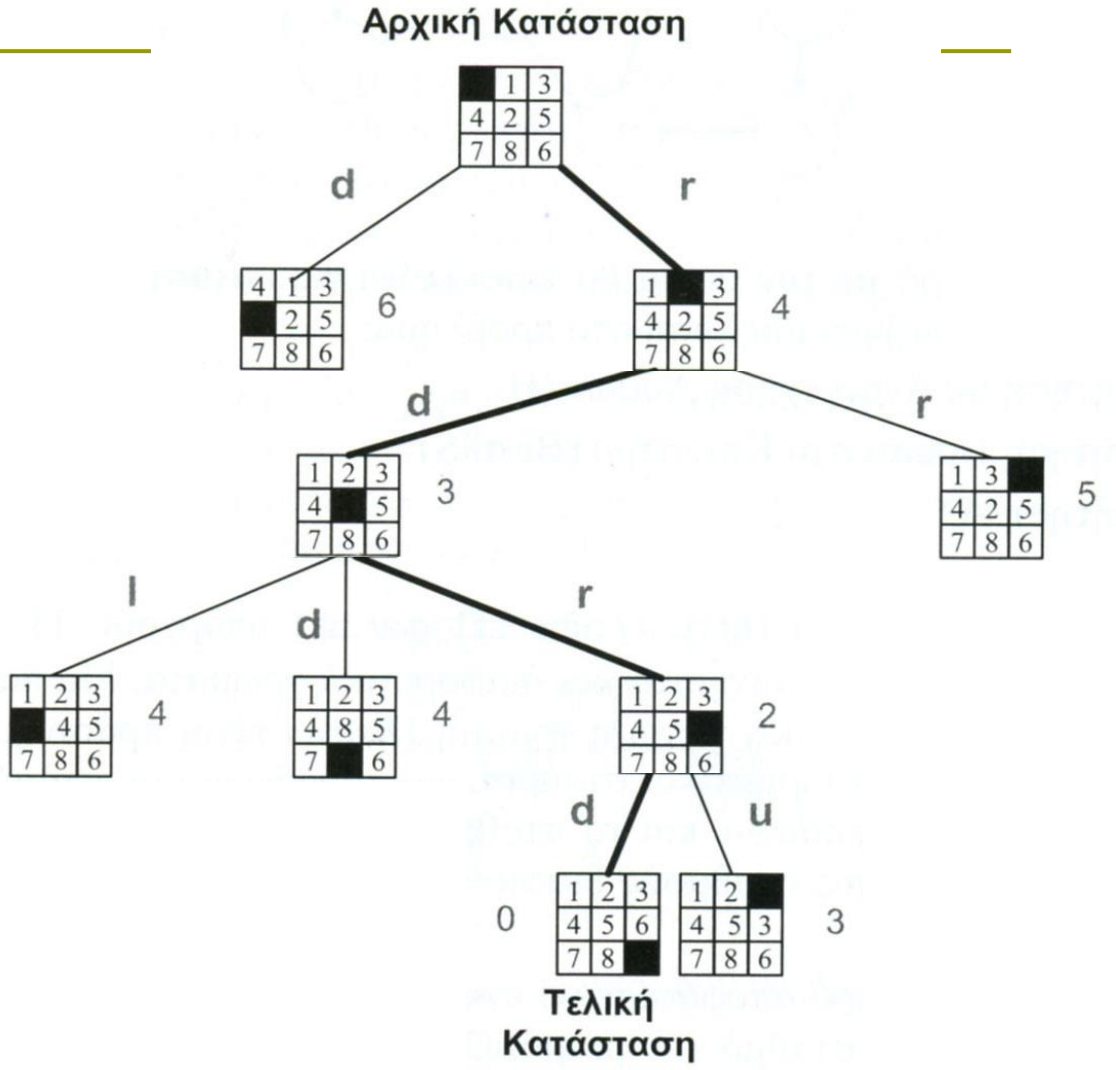


Παράδειγμα BestFS σε 8-puzzle

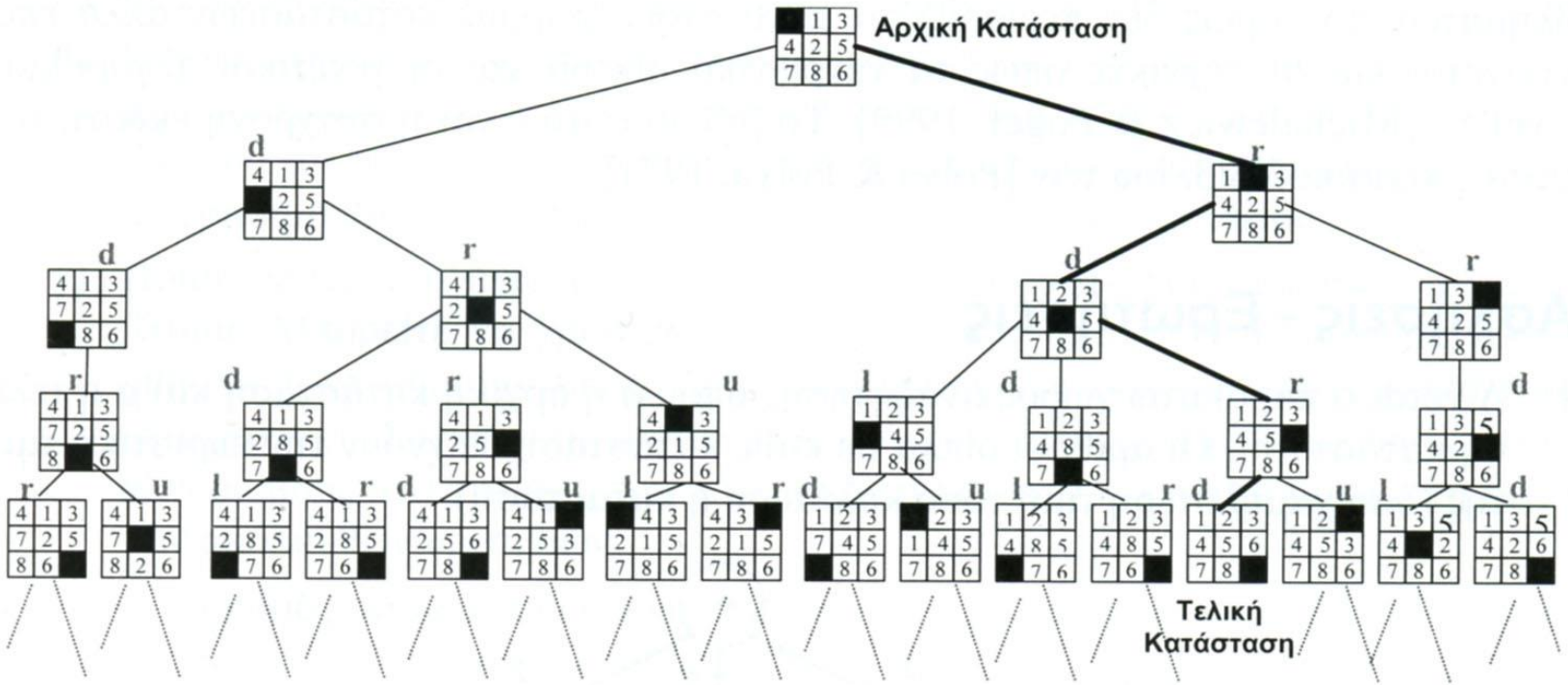
Ευρετική συνάρτηση=

Πλακάκια σε λάθος θέση

Η ευριστική τιμή υπολογίζει πόσα πλακάκια είναι εκτός θέσης συμπεριλαμβανομένου του κενού



Παράδειγμα BestFS σε 8-puzzle
Χώρος Αναζήτησης



Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
- Αλγόριθμοι ευρετικής αναζήτησης
  - Αναζήτηση Πρώτα στο Καλύτερο (BestFS)
  - Αναρρίχηση Λόφων (HC)
  - Αλγόριθμος A*
- Παιχνίδια δύο αντιπάλων

Αναρρίχηση λόφων (Hill-Climbing, HC)

- Η αναρρίχηση λόφων μοιάζει με την αναζήτηση πρώτα-στο-καλύτερο, με τις εξής διαφορές:
  - Δεν έχει σύνορο αναζήτησης.
  - Δεν έχει κλειστό σύνολο
- Μειονεκτήματα:
  - Δεν θυμάται και άρα δεν επιστρέφει να εξετάσει τις εναλλακτικές "διαδρομές".
    - Άρα μπορεί να μην βρει λύση, ακόμη και όταν υπάρχει.
  - Μπορεί να επισκεφθεί πολλές φορές την ίδια κατάσταση
- Πλεονεκτήματα
  - Μηδενικές απαιτήσεις σε μνήμη

Αναρρίχηση λόφων (HC)

- Απλοϊκή στρατηγική **ορειβάτη** σε ομίχλη που προσπαθεί να **φτάσει στην κορυφή**.
  - Ποιο από τα σημεία που βλέπω γύρω μου είναι το πιο ψηλό (και πιο ψηλό από αυτό στο οποίο βρίσκομαι);
  - Πάω προς αυτό.
  - Δεν κρατάω κανένα άλλο γειτονικό σημείο στο μέτωπο.
- Πολύ **οικονομικός σε χρόνο και μνήμη**
  - πάντα μόνο ένας κόμβος στο μέτωπο.
- Προβλήματα:
  - Εγκλωβισμός σε **τοπικό μέγιστο** (τοπική κορυφή).
  - Περιπλάνηση σε **οροπέδιο** (καμία άνοδος ορατή).
  - Γενικά **μη αποδεκτός** και **ατελής**.

Αλγόριθμος αναρρίχησης λόφων

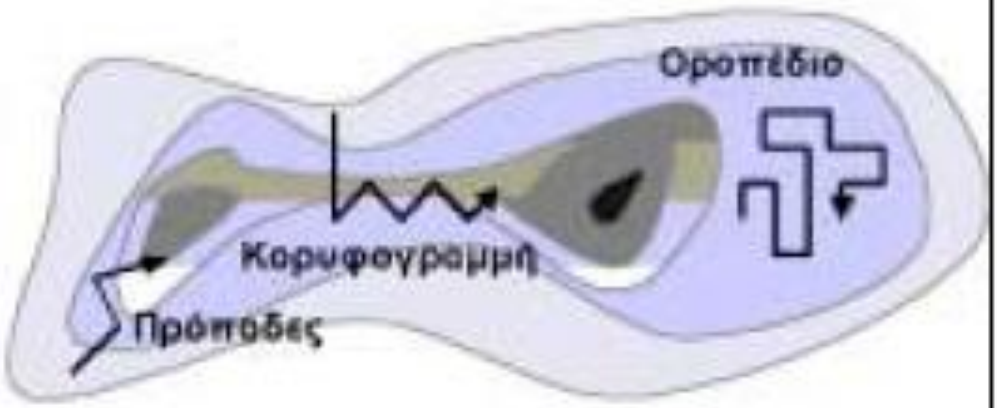
1. Κάνε τρέχουσα κατάσταση την **αρχική**.
2. Αν η τρέχουσα κατάσταση είναι **τελική**, ανάφερε τη λύση και σταμάτησε.
3. **Επέκτεινε** την τρέχουσα κατάσταση και **αξιολόγησε** τις καταστάσεις-παιδιά με την ευριστική συνάρτηση.
4. **Αν δεν υπάρχει παιδί** για το οποίο η τιμή της ευριστικής συνάρτησης να είναι καλύτερη από την τιμή της τρέχουσας κατάστασης, σταμάτησε.
5. Κάνε τρέχουσα κατάσταση το παιδί με την **καλύτερη τιμή της ευριστικής συνάρτησης**.
6. Πήγαινε στο βήμα 2.

Ο αλγόριθμος HC (Ψευδοκώδικας)

```
algorithm hc(InitialState, FinalState)
    begin
        CurrentState←InitialState;
        while CurrentState≠FinalState do
                Children←Expand(CurrentState);
                if Children=∅ then return failure;
                EvaluatedChildren←Heuristic(Children);
                bestChild←best(EvaluatedChildren);
                if hValue(CurrentState) ≥ hValue(bestChild)
                        then return failure;
                        else CurrentState←bestChild;
                endif;
        endwhile;
    return success;
end.
```

Αναρρίχηση λόφων (Hill-Climbing, HC)

- Βασικά προβλήματα του HC:
  - Λόφοι (hills).
  - Οροπέδιο (plateau).
  - Κορυφογραμμή (ridges).
- Βελτιώσεις:
  - Τυχαίες επανεκινήσεις (Random restarts)
  - Εξαναγκασμένη αναρρίχηση λόφου (Enforced Hill-Climbing - EHC)
  - Προσομοιωμένη εξέλιξη (Simulated Annealing - SA)
  - Αναζήτηση με απαγορευμένες καταστάσεις (Tabu Search - TS).

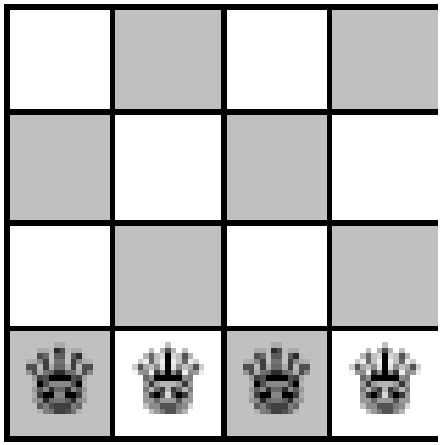


Παραλλαγές του HC

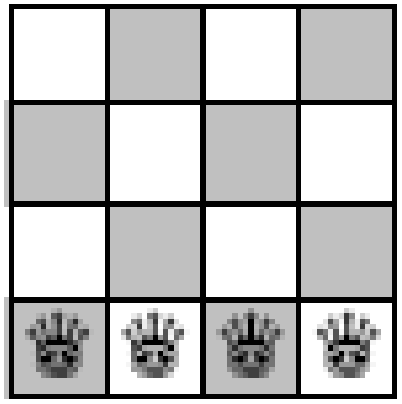
- **Εξαναγκασμένη αναρρίχηση** (EHC)
  - Αν δε βρίσκουμε καλύτερη γειτονική κατάσταση, αναζήτηση σε πλάτος (BFS) μέχρι να βρούμε.
- **Προσομοιωμένη εξέλιξη** (simulated annealing)
  - Επιλέγουμε **τυχαία επόμενη** κατάσταση s'.
  - Αν $\Delta = h(s') - h(s) \leq 0$, μεταβαίνουμε στην s'.
  - Διαφορετικά **μεταβαίνουμε με πιθανότητα** $e^{-\Delta/t}$, όπου t η «θερμοκρασία» (ευνοεί τη μετάβαση). Με πιθανότητα $1- e^{-\Delta/t}$ επιστρέφουμε στην επιλογή άλλης τυχαίας επόμενης κατ/σης.
  - Μιμείται στοιχεία της φυσικής εξέλιξης (κρυσταλλικές δομές).
- **Beam search** (BS)
  - Όπως ο HC αλλά κρατάμε στο μέτωπο τις **k καλύτερες καταστάσεις** (k σταθερά) αντί για μόνο μία.

Το πρόβλημα των βασιλισσών

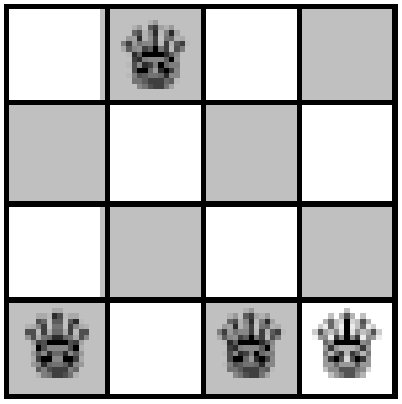
- Θέλουμε να βάλουμε 4 βασίλισσες σε μια σκακιέρα 4x4, έτσι ώστε καμία να μην απειλεί τις άλλες.
- Προφανώς σε κάθε στήλη θα υπάρχει μία μόνο βασίλισσα.
- Ξεκινάμε με όλες τις βασίλισσες στην κάτω γραμμή.
- Σε κάθε βήμα μπορούμε να μετακινήσουμε μια βασίλισσα σέ μια άλλη θέση στη στήλη της, άρα οι δυνατές κινήσεις είναι 4x3=12.
- Χρησιμοποιούμε ως ευρετική συνάρτηση το πόσες απειλές υπάρχουν κάθε φορά (όλες οι απειλές είναι διπλές, εμείς τις μετράμε ως μία απειλή κάθε φορά).



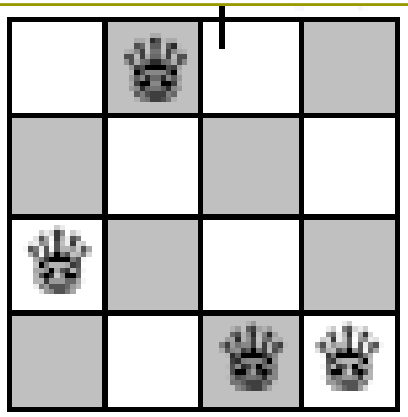
Το πρόβλημα των βασιλισσών



6

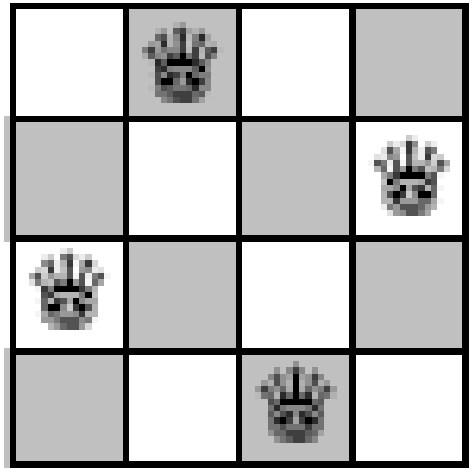


3



1

- . Στο συγκεκριμένο παράδειγμα είμασταν τυχεροί, μιας και όλες οι επιλογές μας βγήκαν σωστές.
- . Δοκιμάστε να λύσετε το ίδιο πρόβλημα σε σκακιέρα 8x8 με 8 βασίλλισες.



0

Επίλυση Προβλημάτων

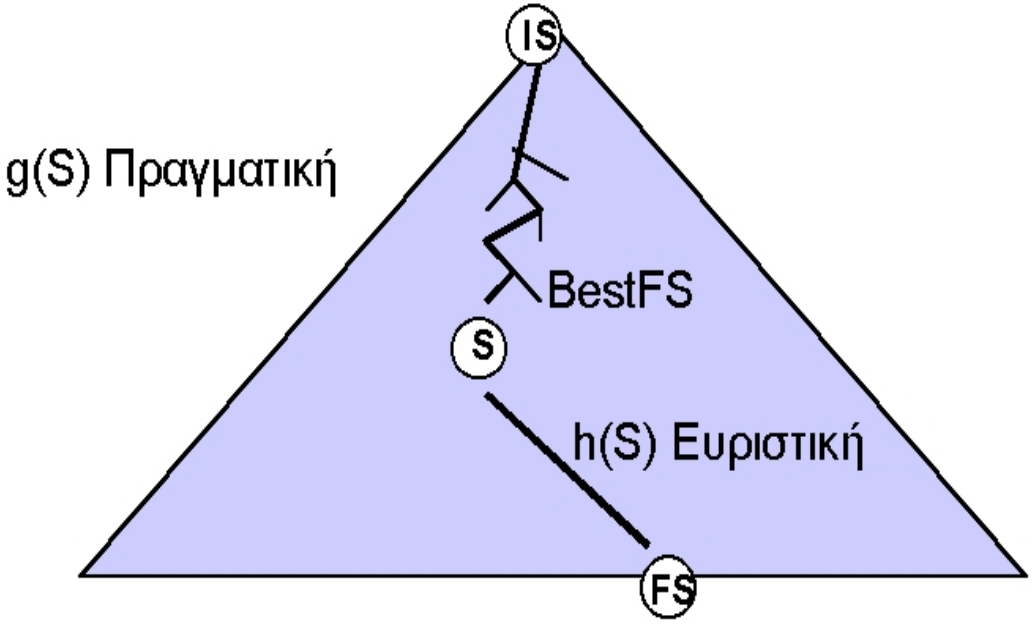
- Περιγραφή Προβλημάτων
- Αλγόριθμοι αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
- Αλγόριθμοι ευρετικής αναζήτησης
  - Αναζήτηση Πρώτα στο Καλύτερο (BestFS)
  - Αναρρίχηση Λόφων (HC)
  - Αλγόριθμος A*
- Παιχνίδια δύο αντιπάλων

Ο αλγόριθμος **(A-star)** A*

- Ο αλγόριθμος A* είναι ίδιος με την αναζήτηση πρώτα-στο-καλύτερο, με την εξής διαφορά:
  - Η ευρετική συνάρτηση στον A* αποτελείται από το άθροισμα της εκτίμησης της απόστασης της τρέχουσας κατάστασης από το τέλος και τη γνωστή απόσταση της τρέχουσας κατάστασης από την αρχική.
- **F(S)=g(S)+h(S)**, όπου:
  - S : η τρέχουσα κατάσταση
  - g(S) : Η απόσταση της τρέχουσας κατάστασης από την αρχική.
  - h(S) : Η εκτίμηση της απόστασης της τρέχουσας κατάστασης από το τέλος.

Ο αλγόριθμος A*

- Ίδιος με τον BestFS αλλά η ευριστική συνάρτηση είναι **f(s) = h(s) + g(s)**.
- **h(s)**: όπως στον BestFS (εκτίμηση της απόστασης που απομένει).
- **g(s)**: η απόσταση ως τώρα από τη ρίζα.



Σημαντική ιδιότητα

- Αν για κάθε κατάσταση η τιμή h(S) είναι μικρότερη ή το πολύ ίση με την πραγματική απόσταση της S από την τελική κατάσταση, τότε ο A* βρίσκει πάντα τη βέλτιστη λύση.
  - Ευρετικές συναρτήσεις που επιστρέφουν πάντα εκτιμήσεις μικρότερες της κανονικής (υπο-εκτιμήσεις) ονομάζονται **αποδεκτές συναρτήσεις** (admissible functions).

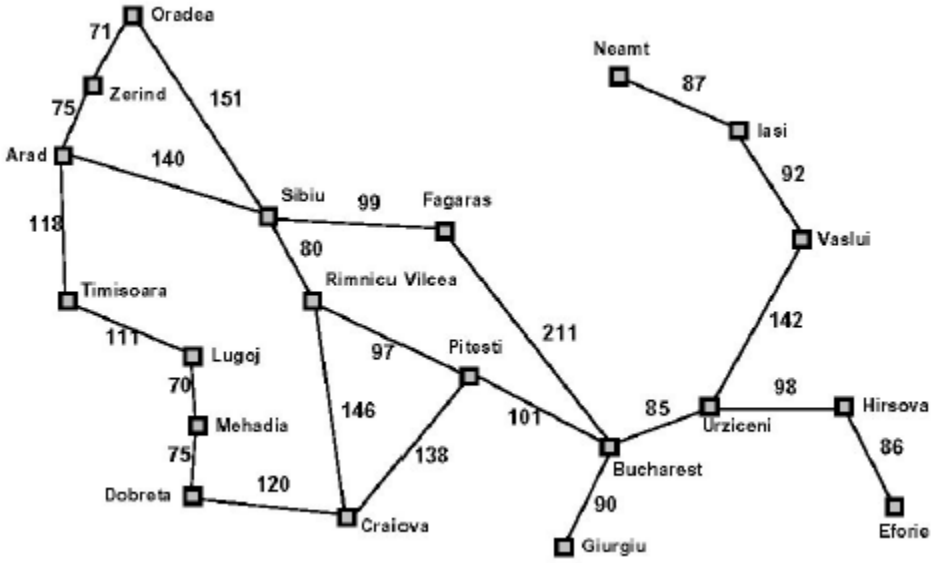
A* Αναζήτηση

- A* λειτουργεί όπως ο BestFS αλλά επεκτείνει τον κόμβο με το μικρότερο f(n)=h(n)+g(n)
- Πλήρης αν το b είναι πεπερασμένο και το κόστος κάθε μετάβασης είναι >0
- Βέλτιστος αν η h είναι αποδεκτή
  - Αποδεκτή ευριστική συνάρτηση: $h(n) \leq C(n)^*$
  - $C(n)^*$ Το πραγματικό κόστος του βέλτιστου μονοπατιού

Πρόβλημα

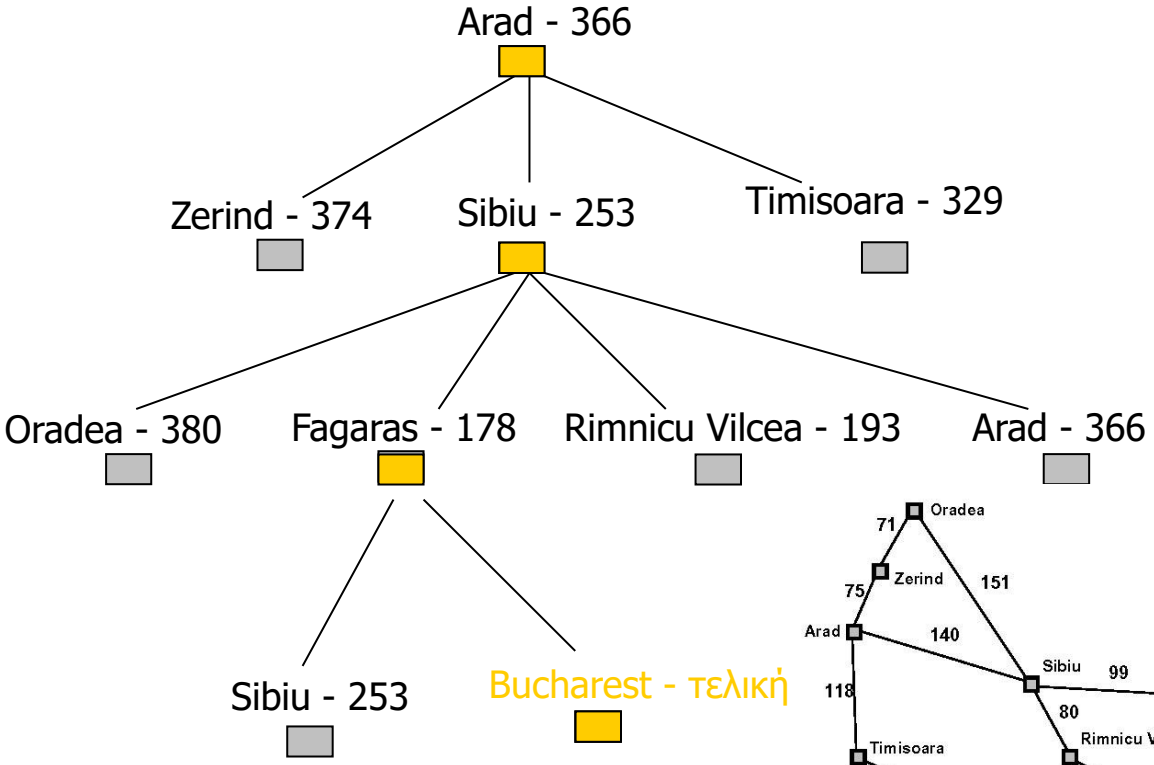
Στο οδικό δίκτυο, προσπαθούμε να βρούμε με τον αλγόριθμο αναζήτησης πρώτα του καλύτερου (BestFS) ένα μονοπάτι από το Arad στο Βουκουρέστι. Οι ακμές παριστάνουν οδούς και οι ετικέτες των ακμών τα μήκη των αντιστοίχων οδών. Χρησιμοποιούμε ως ευριστική συνάρτηση την ευθεία απόσταση μέχρι το Βουκουρέστι .

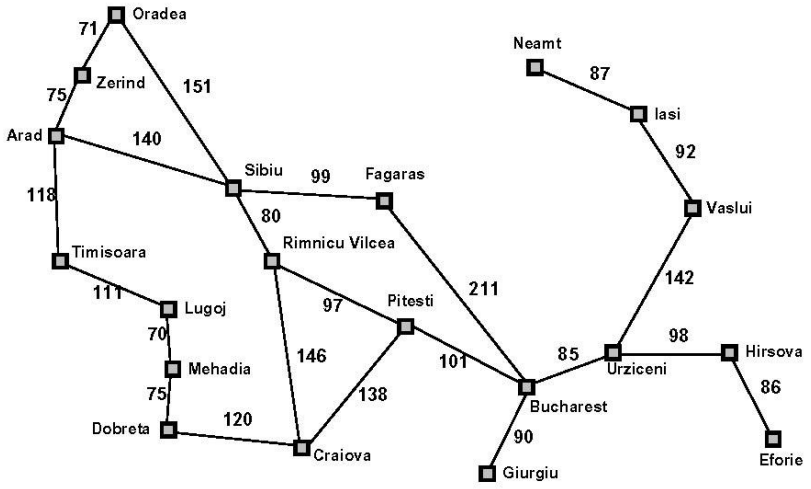
Σχεδιάστε δέντρο αναζήτησης που να δείχνει τους κόμβους που παράγει ο αλγόριθμ μέχρι να βρει το πρώτο μονοπάτι και τις αντίστοιχες τιμές της ευριστικής συνάρτησης.



Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Λύση (Αλγόριθμος BestFS)



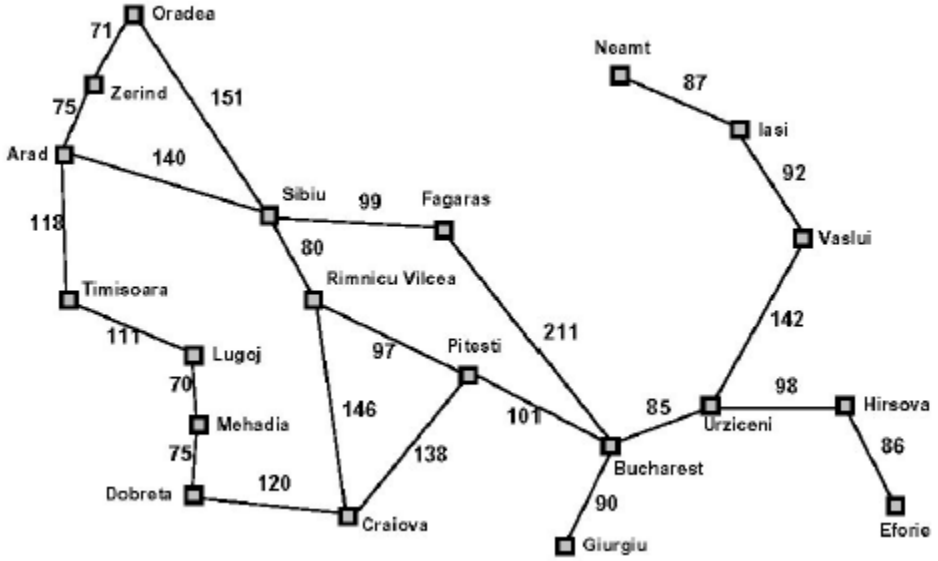


Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Πρόβλημα

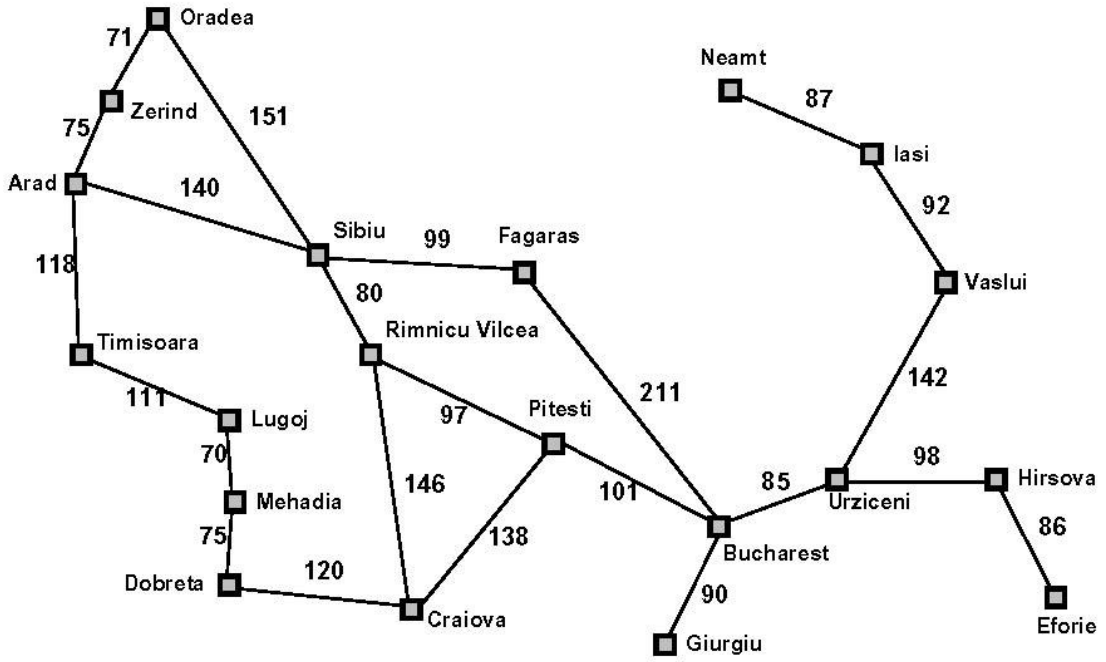
Χρησιμοποιούμε τον αλγόριθμο Α* (χωρίς κλειστό σύνολο) για να βρούμε στον παρακάτω γράφο ένα μονοπάτι από το Arad στο Βουκουρέστι. Οι ακμές παριστάνουν οδούς και οι ετικέτες των ακμών τα μήκη των αντιστοίχων οδών. Χρησιμοποιούμε ως ευριστική συνάρτηση την ευθεία απόσταση μέχρι το Βουκουρέστι (βλ. πίνακα).

Σχεδιάστε δέντρο αναζήτησης που να δείχνει όλους τους κόμβους (και μόνο αυτούς) που εξερευνά ο αλγόριθμος μέχρι να επιστρέψει το πρώτο μονοπάτι που εντοπίζει.Το δέντρο να δείχνει και πώς αξιολογεί κάθε κόμβο ο αλγόριθμος.



Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Απάντηση (Αλγόριθμος A*)

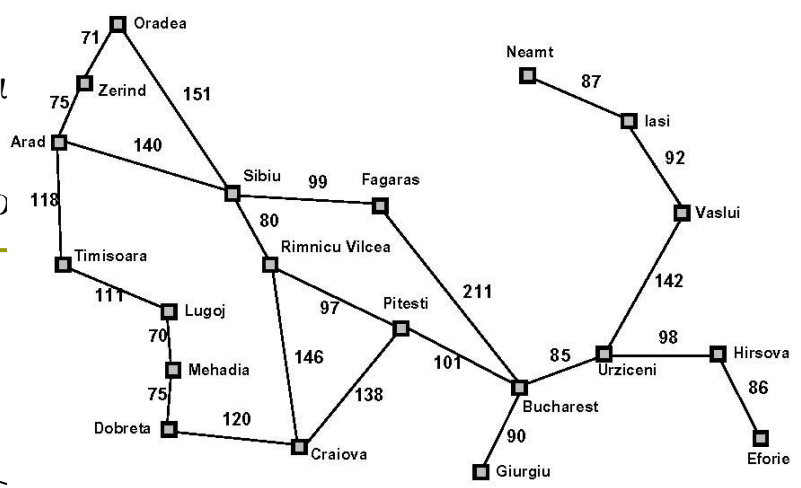


Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

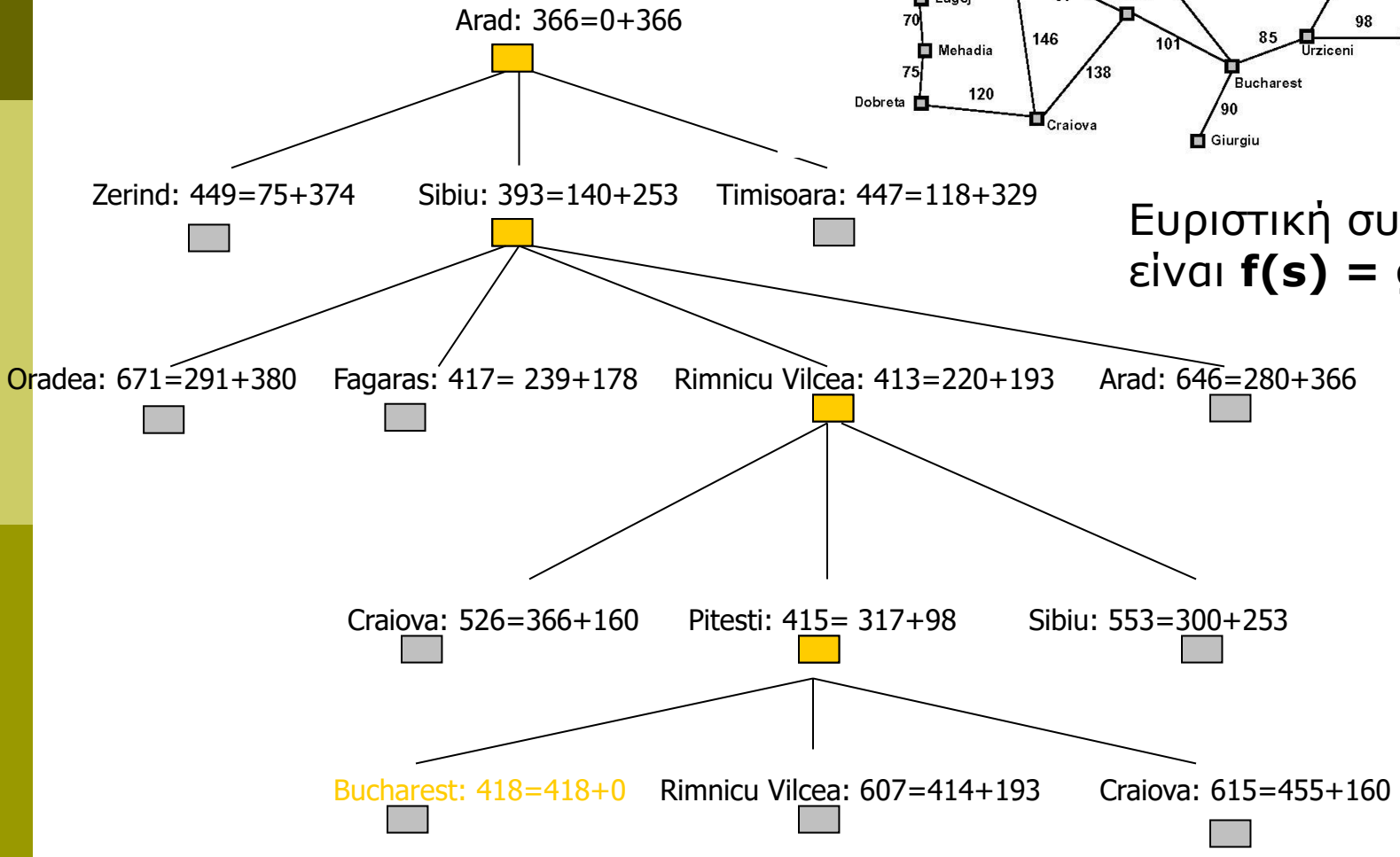
Ο Αλγόριθμός A* είναι ίδιος με τον BestFS αλλά η ευριστική συνάρτηση είναι **f(s) = g(s)+ h(s)**.

Αλγόριθμος (A*)

Σχεδιάστε δέντρο αναζήτησης που να δείχνει όλους τους κόμβους (και μόνο αυτούς) που εξερευνά ο αλγόριθμος μέχρι να επιστρέψει το πρώτο μονοπάτι που εντοπίζει.



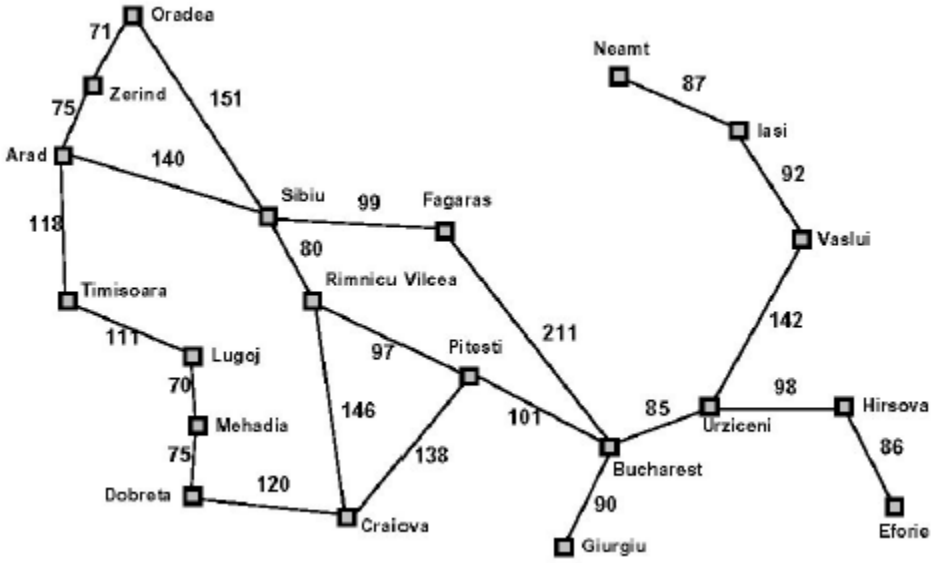
Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



Ευριστική συνάρτηση είναι **f(s) = g(s)+ h(s)**

Πρόβλημα

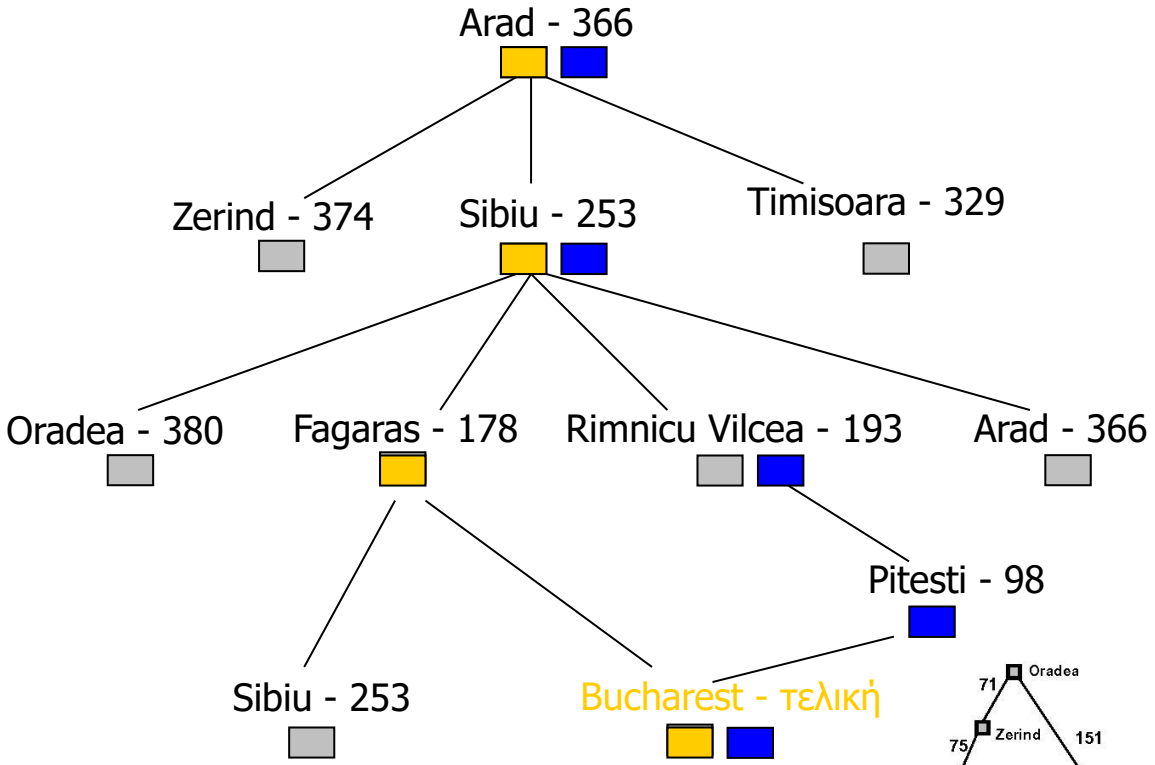
Δείξτε με ένα
παράδειγμα ότι ο BestFS
δεν βρίσκει πάντα πρώτα το βέλτιστο (συντομότερο) μονοπάτι.

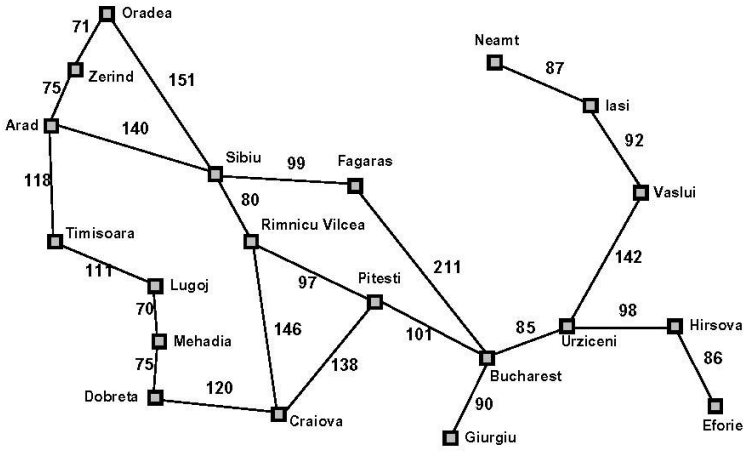


Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Λύση

Δείξτε με ένα παράδειγμα ότι ο BestFS δεν βρίσκει πάντα πρώτα το βέλτιστο (συντομότερο) μονοπάτι.

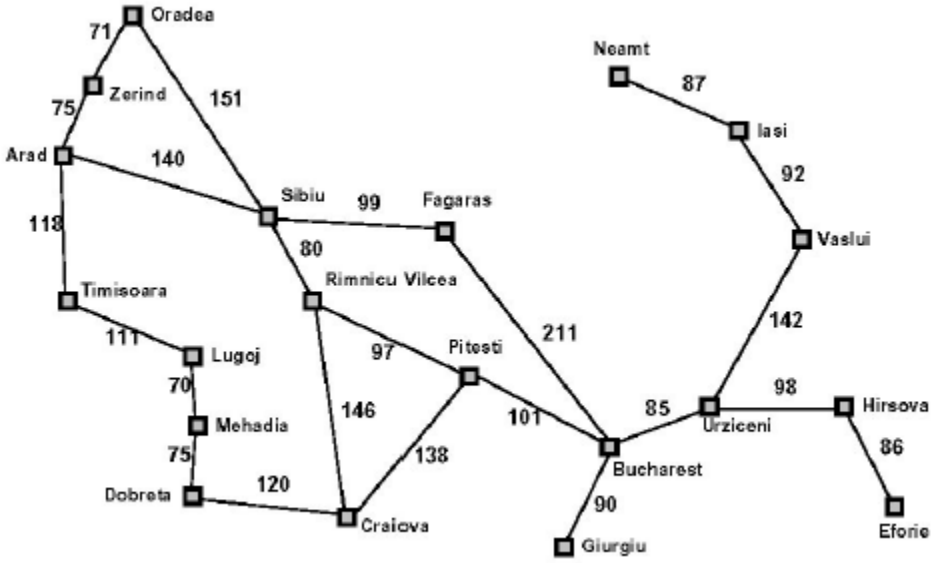




Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Πρόβλημα

Εξηγήστε τι θα συμβεί στην περίπτωση που δεν χρησιμοποιούμε κλειστό σύνολο, αν επιχειρήσουμε να πάμε από το Ιάσιο στο Fagaras και η ευθεία απόσταση Neamt-Fagaras είναι μικρότερη από την ευθεία απόσταση Vaslui-Fagaras.

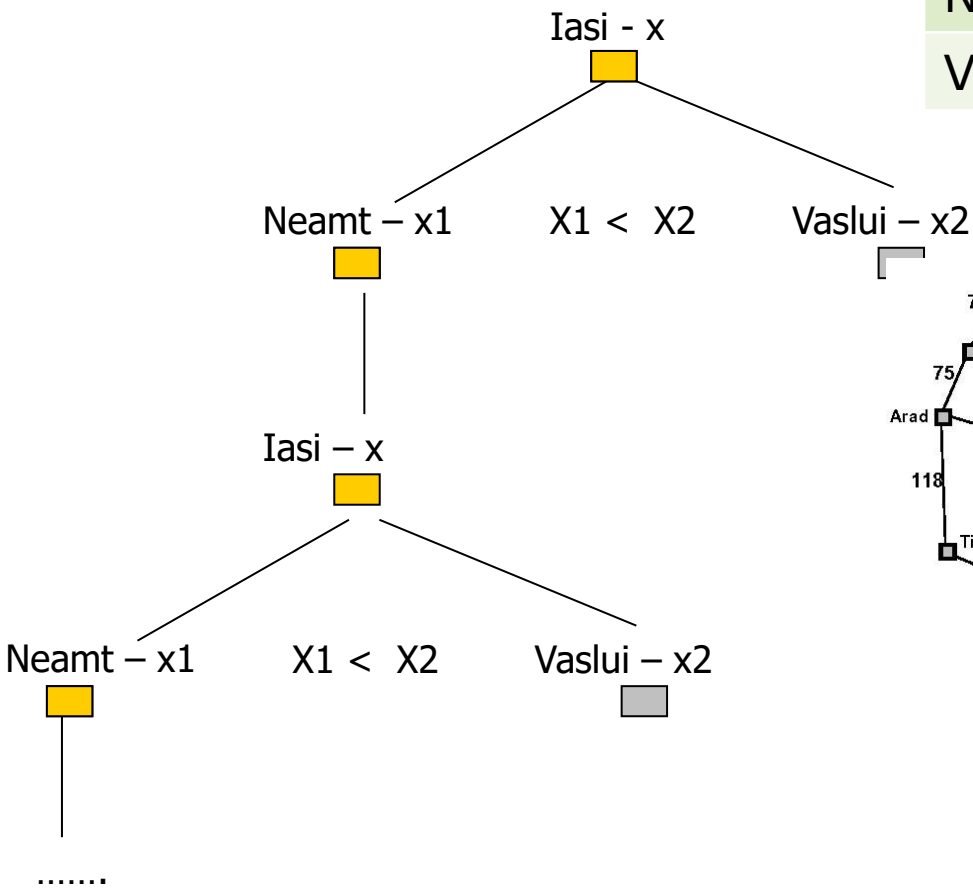


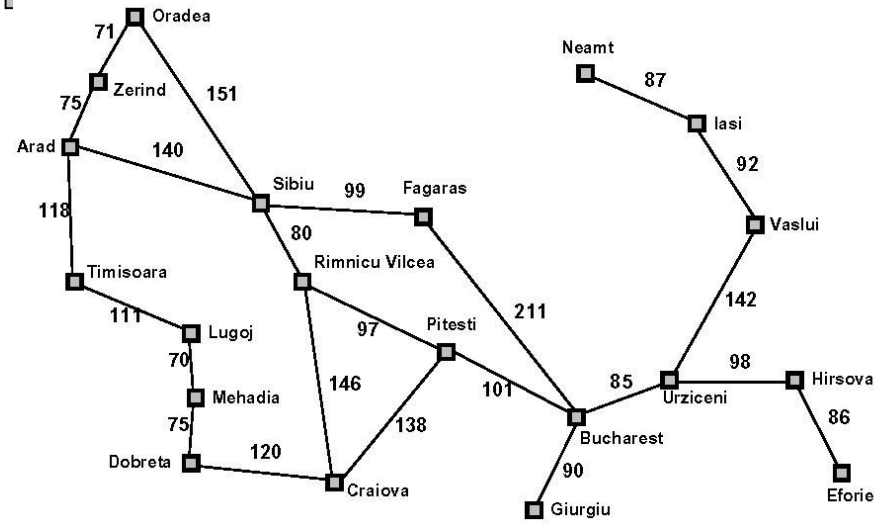
Straight−line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Λύση (γ)

Εξηγήστε τι θα συμβεί στην περίπτωση που δεν χρησιμοποιούμε κλειστό σύνολο, αν επιχειρήσουμε να πάμε από το Ιάσιο στο Fagaras και η ευθεία απόσταση Neamt Fagaras είναι μικρότερη από την ευθεία απόσταση Vaslui-Fagaras.

Απόσταση	
Neamt- Fagaras	X1=120Km
Vaslui-Fagaras.	X2=130Km



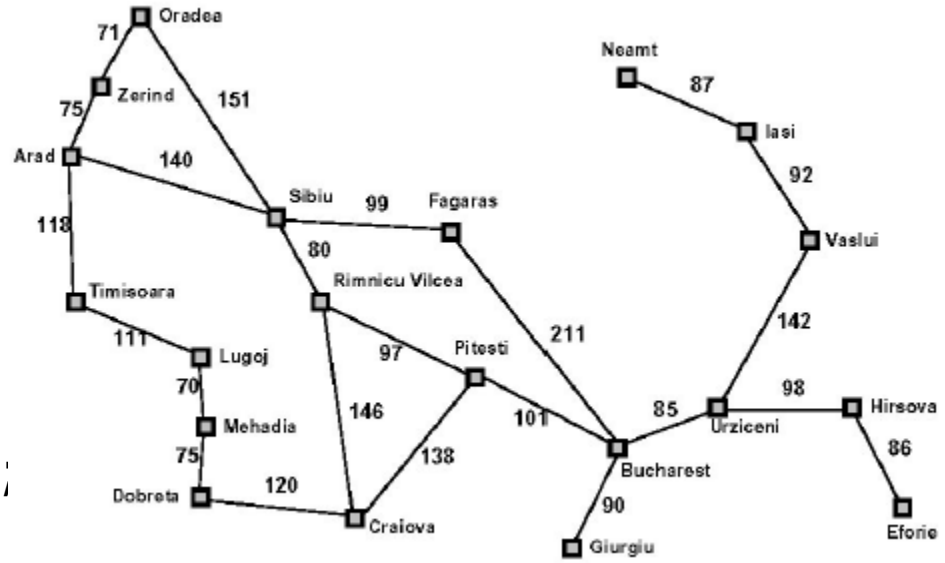


Πρόβλημα

Χρησιμοποιούμε τον αλγόριθμο Α* (χωρίς κλειστό σύνολο) για να βρούμε στον παρακάτω γράφο ένα μονοπάτι από το Arad στο Βουκουρέστι. Οι ακμές παριστάνουν οδούς και οι ετικέτες των ακμών τα μήκη των αντιστοίχων οδών. Χρησιμοποιούμε ως ευριστική συνάρτηση την ευθεία απόσταση μέχρι το Βουκουρέστι (βλ. πίνακα).

(β) Είναι αποδεκτή η ευριστική συνάρτηση που χρησιμοποιούμε και γιατί; Τι μας εξασφαλίζει αυτό;

(γ) Είναι συνεπής η ευριστική συνάρτηση που χρησιμοποιούμε και γιατί; Σε τι θα μπορούσε να μας χρησιμεύσει αυτό;



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

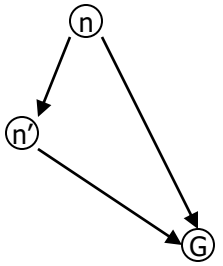
Απάντηση (β,γ)

- Η ευριστική:
  - Είναι αποδεκτή γιατί η ευθεία απόσταση που ενώνει δύο σημεία είναι μικρότερη από οποιαδήποτε άλλη, άρα $h(n) \leq C(n)^*$ για κάθε n.
  - Είναι συνεπής.

  Απόδειξη: Έστω δύο πόλεις n,n′, με ευρηστικές συν/σεις h(n), h(n′). Συμβολίζουμε με cl(n,n′) την ευθεία απόσταση μεταξύ των πόλεων και από βασικές ιδιότητες της Ευκλείδειας μετρικής έχουμε ότι $h(n) \leq cl(n,n') + h(n')$ (1)

  Επιπλέον έχουμε ότι $cl(n,n') \leq c(n,a,n')$ (2)

  Από (1),(2) έχουμε ότι $h(n) \leq c(n,a,n') + h(n')$



Επιδόσεις Αλγορίθμων Αναζήτησης

Κριτήριο / Αλγόριθμος Αναζήτησης	Πλήρης	Βέλτιστος	Πολ/τητα Χρόνου	Πολ/τητα Χώρου
BFS	Ναι, αν b πεπερασμένο.	Ναι, αν κόστος αύξουσα συν/ση του βάθους.	$O(b^{d+1})$	$O(b^{d+1})$
DFS	Όχι.	Όχι, αν κόστος αύξουσα συν/ση του βάθους.	$O(b^{m})$	$O(bm)$
IDS	Ναι, αν b πεπερασμένο.	Ναι, αν κόστος αύξουσα συν/ση του βάθους.	$O(b^{d})$	$O(bd)$
BestFS	Όχι.	Όχι.	$O(b^{m})$	$O(b^{m})$

Ερωτήσεις επιδόσεων

- Συμφωνείτε ή Διαφωνείτε?

• Όταν χρησιμοποιείται κλειστό σύνολο, ο DFS αποκλείεται να παγιδευθεί σε άπειρα μονοπάτια.

• Με χρήση κλειστού συνόλου χάνεται το πλεονέκτημα της γραμμικής πολυπλοκότητας χώρου του DFS.

• Η χρήση κλειστού συνόλου στον BFS δεν αλλάζει την πολυπλοκότητα χώρου του αλγορίθμου.

• Η χρήση κλειστού συνόλου στον IDS δεν αλλάζει την πολυπλοκότητα χώρου του αλγορίθμου.

Επίλυση Προβλημάτων

- Περιγραφή Προβλημάτων
- Αλγόριθμοι αναζήτησης
- Αλγόριθμοι τυφλής αναζήτησης
- Αλγόριθμοι ευρετικής αναζήτησης
  - Αναζήτηση Πρώτα στο Καλύτερο (BestFS)
  - Αναρρίχηση Λόφων (HC)
  - Αλγόριθμος A*
- Παιχνίδια δύο αντιπάλων